Brac University

Department of Electrical & Electronic Engineering

Fall 2026

Course Number : EEE343L

Section :01

Group No :02

Lab Report

Experiment no: 3

Name of the experiment:

Introduction to Discrete-time signals and different operations performed on these signals using MATLAB

Name: Souvik Barman Ratul ID: 24121205

All Group members:

Sl	*ID*	*Name*	Signature
1.	24121205	Souvik Barman Ratul	Ratul
2.	23221051	Syed Abrar Faiaz	Faiaz
3.	23221052	Sadman Shabab Hossain	Shabab
4.	23221040	Afifa Tanzim Ahona	Ahona
5.	23221035	Tuli Saha	Tuli

Date of Submission: 02/11/2026

Objective:

Performing different operations such as shifting, folding, addition on discrete time signals using MATLAB and observing output

Lab codes:

Problem 1- Write a function to produce **unit impulse** sequence.

Function

```
function [n,x]=impseq(n_start, n_end, n0)
n= n_start : n_end;
x= zeros(1,length(n));
x(n==n0) =1;
end
```

Main Code

```
%% unit impulse sequence
n0=-1;
[n,x]=impseq(-2,5,n0)
figure('color','w');
prettystem(n,x, sprintf('Unit Impulse \\delta(n-%d)',n0));
```

Output

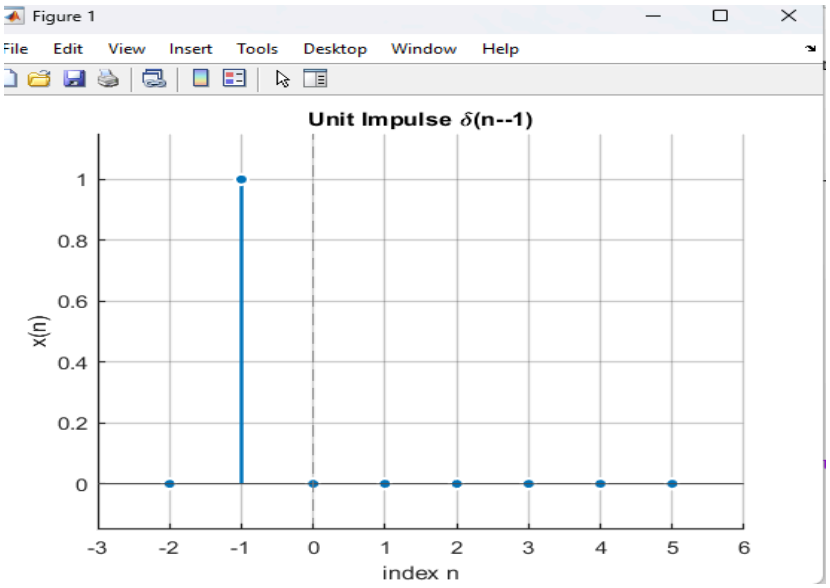


Figure:- Index n= -1

Unit Step

Function

```
function [n,x]=unitseq(n_start, n_end, n0)
n= n_start : n_end;
x= double(n>=n0);
end
```

Main Code

```
%% unit Step sequence
[n,x]=unitseq(-5,6,-2);
figure('color','w');
prettystem(n,x, 'Unit Step u(n)', [0.85 0.33 0.10]);
```

Output:

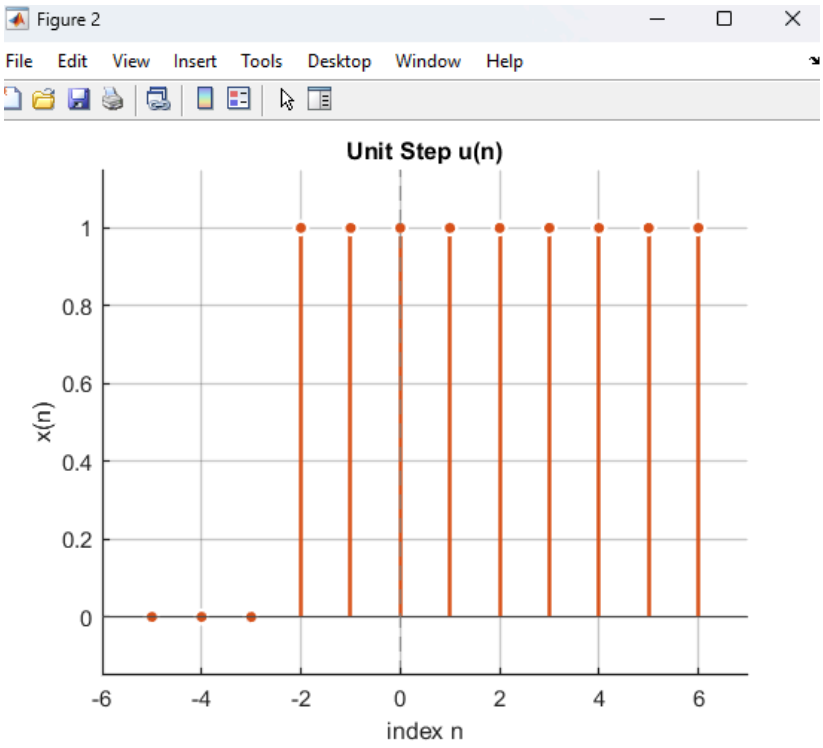


Figure:- Unit step graph

Ramp Signal:

Code

```
%% Ramp Signal
n= -5:5;
x =(n>=2).*n;
figure('color','w');
prettystem(n,x, 'Ramp r(n)', [0.47 0.67 0.19]);
```

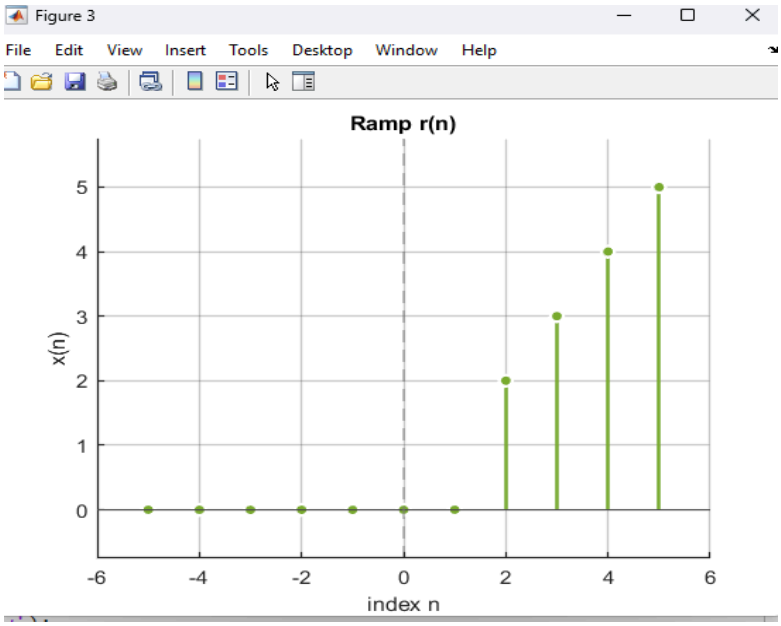


Figure:- Ramp signal graph

Problem 2- Generate: $x(n) = e^{(2+j3)n}$, $0 \leq n \leq 10$

Main Code

```
%%Problem 2: x(n) =e^((2+j3)n)
n=0:10;
x=exp ((2+3j)*n);
figure('Color','w');
subplot(3,1,1);prettystem(n,real(x),'Real part(linear scale)');
ylabel('Re\{x(n)\}');
subplot(3,1,2);prettystem(n,imag(x),'Imaginary part(linear scale)',[0.0 0.85 0.33]);
ylabel('Im \{x(n)\}');
subplot (3,1,3);prettystem(n,abs(x),'Magnitude (linear scale)', [0.47 0.67 0.19]);
ylabel('|x(n)|');
```

Output

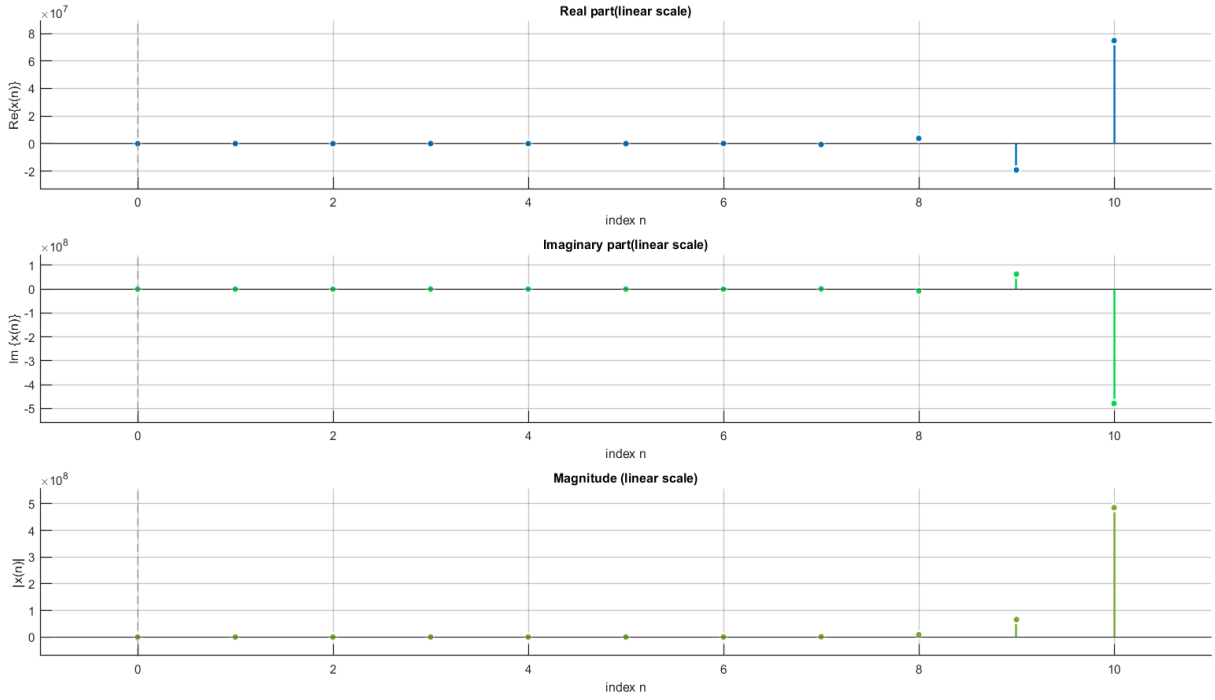


Figure:- Graph in 3 different way (Real,Imaginary and magnitude)

Problem 3- Add the following two signals: x1(n) = {2, 5, 4, -3, 4, 2} and x2(n) = {1, 2, 4, -5, 3}
↑ ↑

Function

```
function [n,y] = sigadd(n1, x1, n2, x2)
  n = min(min(n1),min(n2)) : max(max(n1),max(n2));
  x1e = zeros(1,length(n));
  x2e = zeros(1,length(n));
  x1e(n>=min(n1) & n<=max(n1)) = x1;
  x2e(n>=min(n2) & n<=max(n2)) = x2;
  y = x1e + x2e;
end
```

Main Code

```
%% Problem 3: Addition of two signals
x1 = [2 5 4 -3 4 2];  n1 = -3:2;
x2 = [1 2 4 -5 3];    n2 = 0:4;
% Add the two signals with non-matching time indices
[n, y] = sigadd(n1, x1, n2, x2);
xl = [min(n)-1, max(n)+1];              % Same x-range for all three plots
% Plotting the original and summed signals
figure('Color','w');
subplot(3,1,1); prettystem(n1, x1, 'x_1(n)');                    xlim(xl);
subplot(3,1,2); prettystem(n2, x2, 'x_2(n)', [0.85 0.33 0.10]);        xlim(xl);
subplot(3,1,3); prettystem(n, y, 'x_1(n) + x_2(n)', [0.47 0.67 0.19]);   xlim(xl);
```

Output:

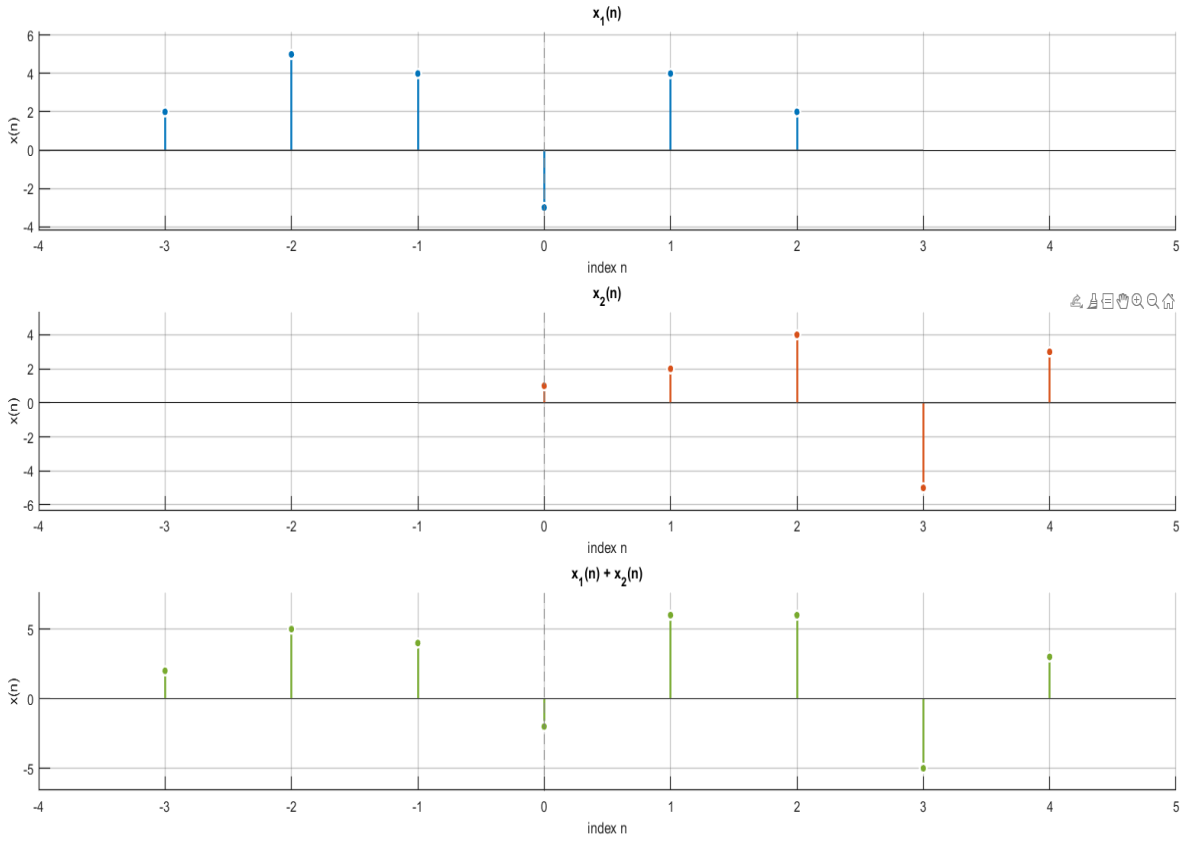


Figure:- Graph of add 2 different signal

Problem 4- Write a function “sigshift” that will take three inputs: the signal, the index and the amount of shift. The function will output the shifted signal and the corresponding index.

Main Code

```
%% Problem 4: Shifting
n = -1:2;
x = [-2 3 5 4];
k = -2;
[m, y] = sigshift(n, x, k);
figure('Color','w');
subplot(2,1,1);
stem(n, x)
subplot(2,1,2);
prettystem(m, y)
```

Output:

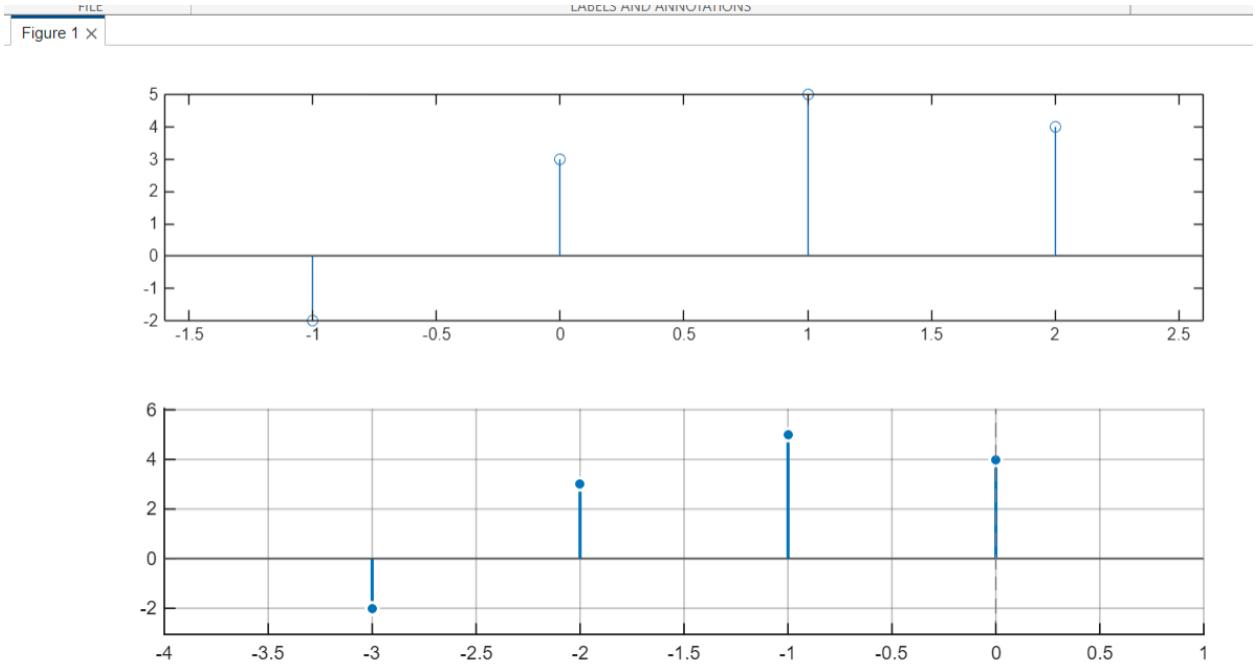


Figure:- Shifting operation graph

Problem 5- Make a function “sigfold” which takes the input sequence x(n) and the input index n as input arguments and gives a folded out y(n) and output index neg_n. Where, y(n) = x(-n). Here, you can use the built-in function “fliplr”.

Function

```
function [m,y] = sigfold(n,x)
m=-fliplr(n);
y= fliplr(x);
end
```

Main Code

```
%% Problem 5: Folding
n = -2:3;
x = [4 -2 3 5 4 6];
[m, y] = sigfold(n, x);
figure('Color','w');
subplot(2,1,1);
stem(n, x)
subplot(2,1,2);
stem(m, y)
```

Output:

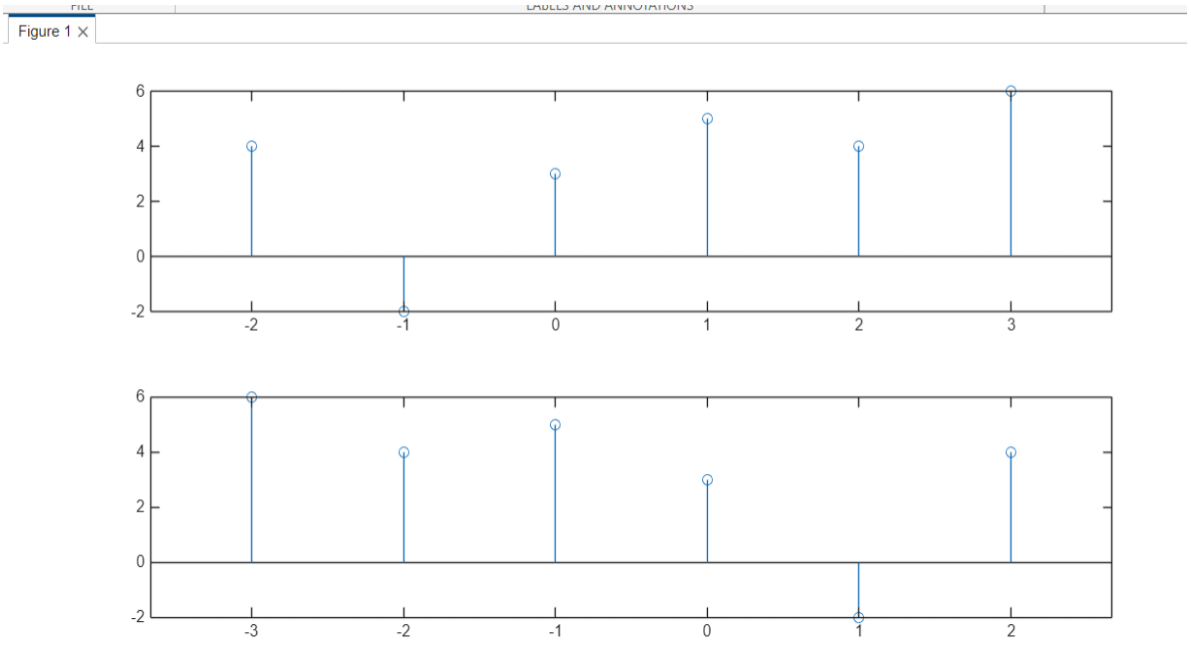


Figure:- Signal fold operation graph

Lab Task:

Write MATLAB code to generate the following-

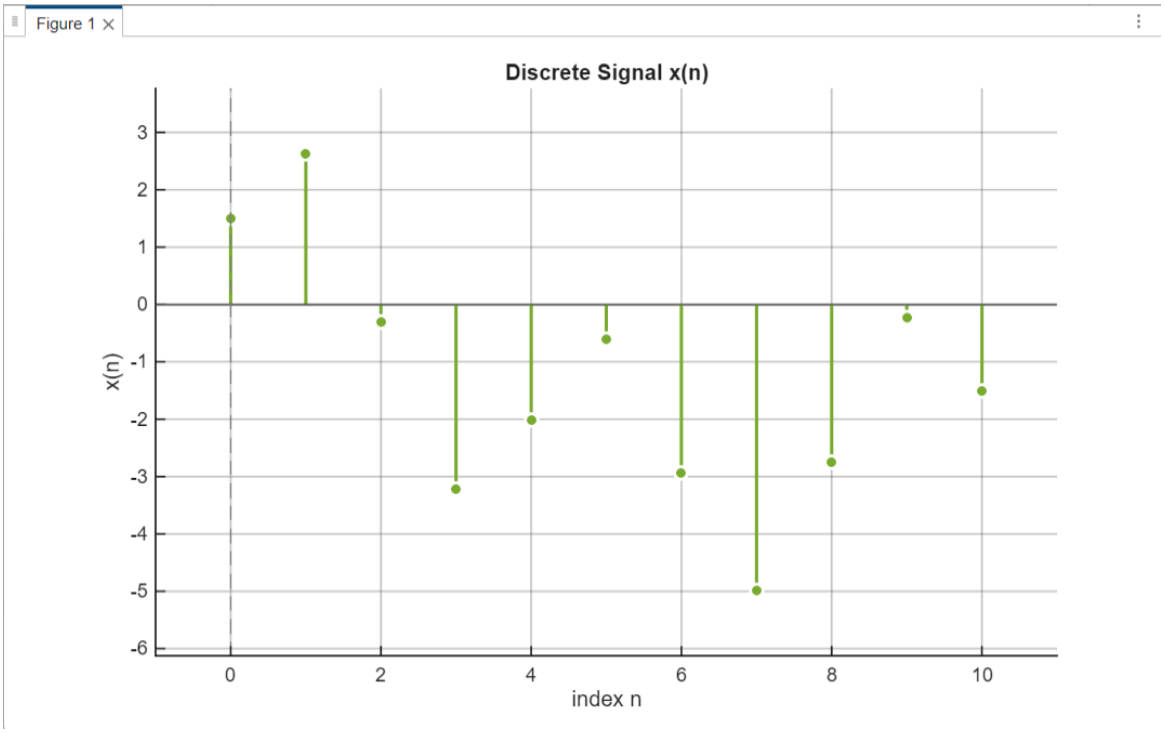
- $x(n) = 3cos(0.1\pi n + \pi/3) + 2sin(0.5\pi n),\ 0 \leq n \leq 10$

Answer

Main Code

```
close all;
clear all;
clc;
n = 0:10;
x = 3*cos(0.1*pi*n + pi/3) + 2*sin(0.5*pi*n);
figure('Color', 'w');
prettystem(n, x, 'Discrete Signal x(n)', [0.47 0.67 0.19]);
```

Output:



Lab Report Assignment:

For generating unit-step and unit-impulse sequences, call the functions that you created during Lab Task.

(i) Generate and plot each of the following sequences over the indicated interval.

a) $x(n) = 2\delta(n+2) - \delta(n-4), \quad -5 \le n \le 5$

b) $x(n) = n[u(n) - u(n-12)] + 10e^{-0.5(n-10)}[u(n-9) - u(n-20)], \quad 0 \le n \le 20$

(ii) Let x(n) = {1, 2, 3, 4, 5, 6, 7, 6, 5, 4, 3, 2, 1} and index = [-5:7]. Determine and plot the following sequences-

a) $x_1(n) = 3x(n-5) - 4x(n+4)$

b) $x_2(n) = x(2-n) + x(n)\,x(n-4)$

Answer:

Function code for Unit Impulse

```
function [n,x]=impseq(n_start, n_end, n0)
n= n_start : n_end;
x= zeros(1,length(n));
x(n==n0) =1;
end
```

Function code for Unit Step

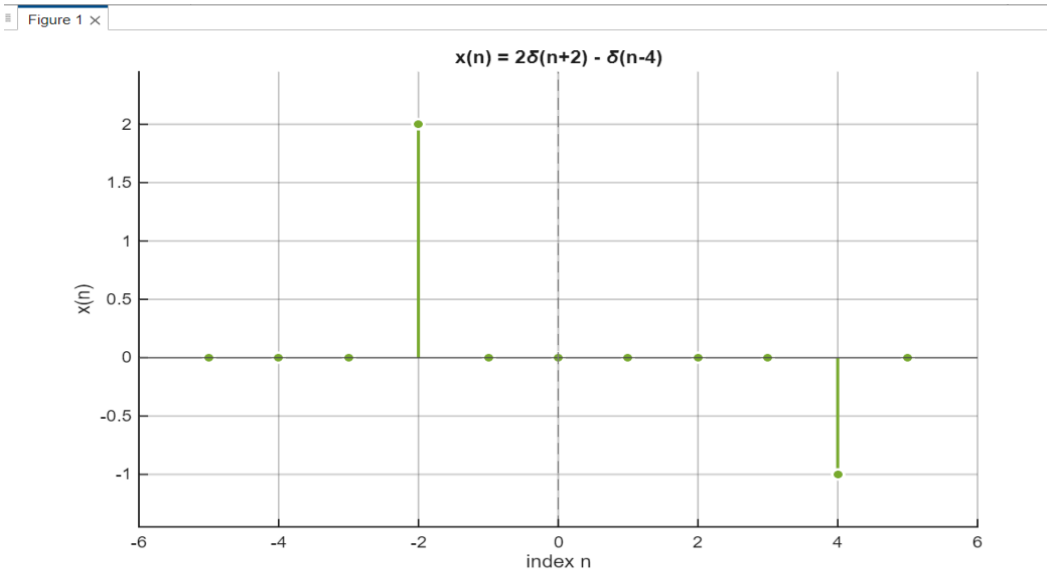
```
function [n,x]=unitseq(n_start, n_end, n0)
n= n_start : n_end;
x= double(n>=n0);
end
```

Main Code:

i(a)

```
%% Question (i(a))
close all;
clear all;
clc;
[n, d1] = impseq(-5, 5, -2);
[~, d2] = impseq(-5, 5, 4);
x = 2*d1 - d2;
figure('Color', 'w');
prettystem(n, x, 'x(n) = 2\delta(n+2) - \delta(n-4)', [0.47 0.67 0.19]);
```

Output:

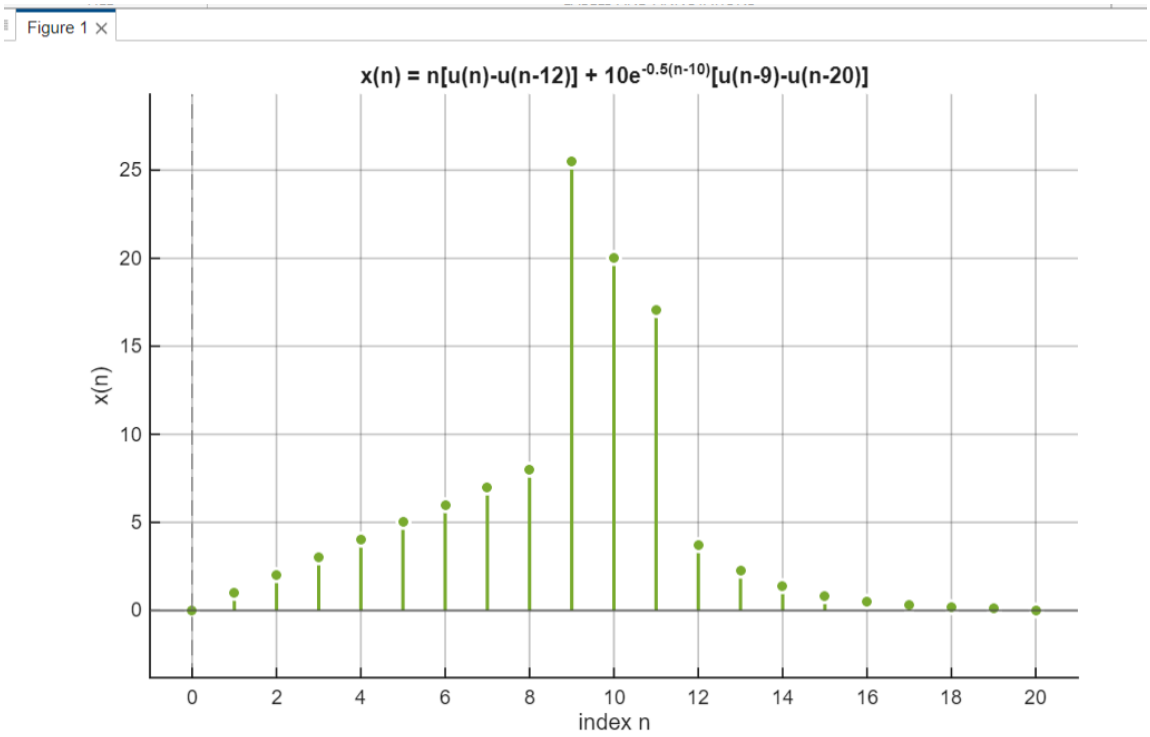


Main Code:

i(b)

```
%% Question (i(b))
close all;
clear all;
clc;
[n, u1] = unitseq(0, 20, 0);
[~, u2] = unitseq(0, 20, 12);
[~, u3] = unitseq(0, 20, 9);
[~, u4] = unitseq(0, 20, 20);
x = n.*(u1 - u2) + 10*exp(-0.5*(n-10)).*(u3 - u4);
figure('Color', 'w');
prettystem(n, x, 'x(n) = n[u(n)-u(n-12)] + 10e^{-0.5(n-10)}[u(n-9)-u(n-20)]', [0.47 0.67
0.19]);
```

Output:



Answer:

Main Code:

(ii)(a)

```
%% Question (ii(a))
close all;
clear all;
clc;

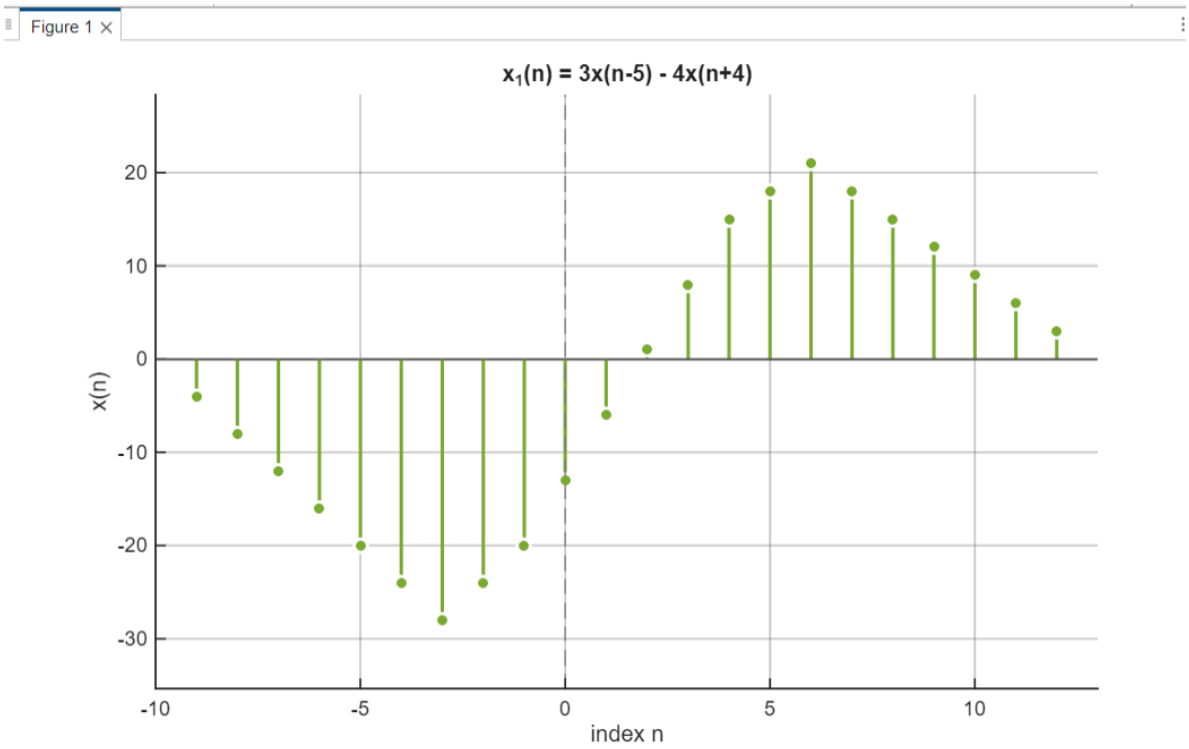
x = [1 2 3 4 5 6 7 6 5 4 3 2 1];
n = -5:7;

[n1, x1_part] = sigshift(n, 3*x, 5);
[n2, x2_part] = sigshift(n, -4*x, -4);

[n_shift, x1] = sigadd(n1, x1_part, n2, x2_part);

figure('Color', 'w');
prettystem(n_shift, x1, 'x_1(n) = 3x(n-5) - 4x(n+4)', [0.47 0.67 0.19]);
```

Output:



(ii)(b)

Function

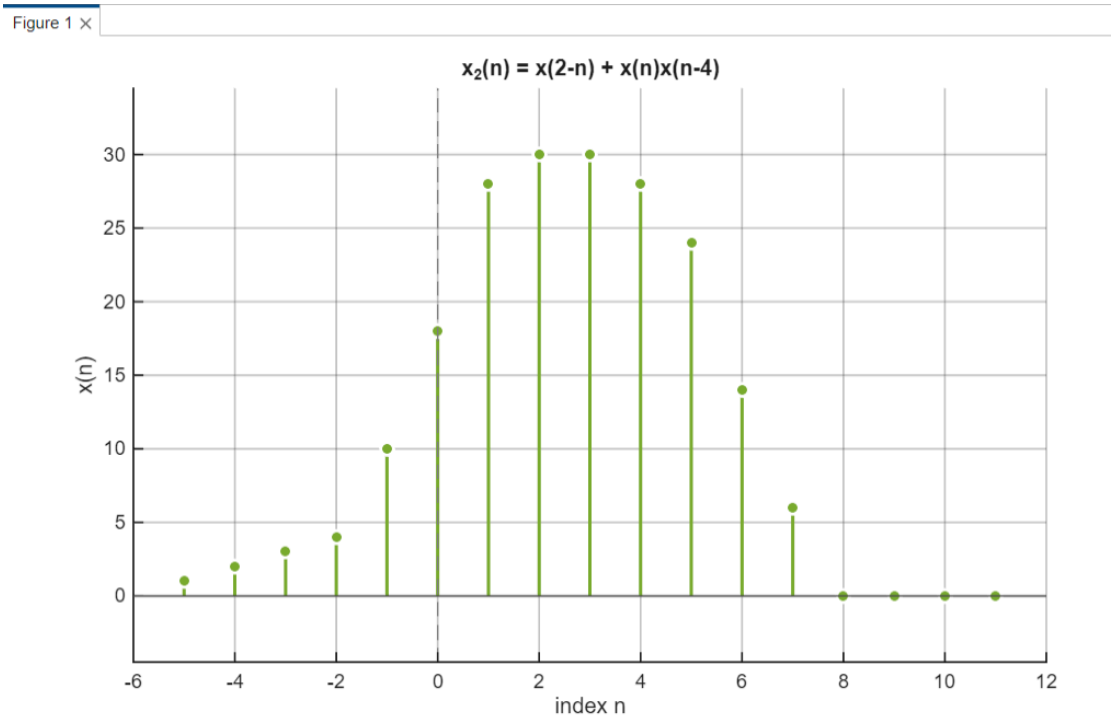
```
function [n, y] = sigmult(n1, x1, n2, x2)
  n = min(min(n1), min(n2)):max(max(n1), max(n2));
  x1e = zeros(1, length(n));
  x2e = zeros(1, length(n));
  x1e(n >= min(n1) & n <= max(n1)) = x1;
  x2e(n >= min(n2) & n <= max(n2)) = x2;
  y = x1e .* x2e;
end
```

Main Code:

```
%% Question (ii(b))
close all;
clear all;
clc;
x = [1 2 3 4 5 6 7 6 5 4 3 2 1];
n = -5:7;

[m_fold, x_fold] = sigfold(n, x);  % x(2-n) -> Fold x(n) and shift by +2
[n_term1, x_term1] = sigshift(m_fold, x_fold, 2);
%  x(n) * x(n-4)
[n_shift4, x_shift4] = sigshift(n, x, 4);
[n_term2, x_term2] = sigmult(n, x, n_shift4, x_shift4);
[n_out, x2] = sigadd(n_term1, x_term1, n_term2, x_term2);
figure('Color', 'w');
prettystem(n_out, x2, 'x_2(n) = x(2-n) + x(n)x(n-4)', [0.47 0.67 0.19]);
```

Output



Discussion:-

In this experiment, basic discrete-time signals such as unit impulse, unit step, ramp, and complex exponential sequences were generated and plotted using MATLAB. Custom functions like “impseq” and “unitseq” were created to generate impulse and step sequences over specified index ranges, while plotting the real part, imaginary part, and magnitude of a complex exponential signal illustrated its behavior across discrete time steps. Fundamental signal operations including shifting, folding, addition, and multiplication were also successfully implemented. When combining signals defined over non-matching time domains, operations like “sigadd” and “sigmult” expanded the time vector to cover both signals by filling undefined regions with zeros, while folding was achieved by inverting the sequence order alongside the time index. Finally, these elementary operations were applied to evaluate composite expressions in the lab assignment and all resulting stem plots aligned with theoretical expectations verifying the correctness of the MATLAB code and index alignment logic.

Conclusion:-

This experiment successfully demonstrated the generation and manipulation of discrete-time signals using MATLAB. By executing operations such as shifting, folding, addition, and multiplication the fundamental concepts of discrete-time signal processing were validated. All output stem plots accurately matched theoretical expectations confirming the reliability of both the MATLAB algorithms and the custom helper functions created for signal analysis.